

Date: July 13, 2014 Hay4Stk16_32: Four stacks, 16-bit data, 32-bit instructions

AdrRAM	simple dual port LUT RAM, black dot is write enable
BlkRAM	single port block RAM with 512x32 shape, write enables for each 16 bits
CCR	single port LUT RAM
Funnel shapes	2:1 or 4:1 multiplexors, data exits at pointed end If has + sign, is an adder if has circled + sign, is XOR logic
Black dots	on RAMs is write enable(s), on registers is load enable
Numbers on left	Expected LUT counts for data paths
Registers	EAR, A0, IR, N, Op2, Op1, ALUctrls
Multiply block	square with * inside, black dot is sign extend input
LUT RAMs	operate in async read mode
Block RAM	operate with no internal output data latches
target clock speed	200MHz on Spartan-6 or Kintex-7 (a lot easier to achieve on Kintex) Block RAM always busy AdrRAM and address ALU always busy data ALU usage much lower

Date: Jan. 3, 2015 Hay4Stk16_32

Control logic implementation strategy

- separate decode circuits for cycles 2, 3, 5 and 6
- cycle 6 (data ALU & CCR controls) registered immediately and register directly drives ALU & CCR update
- remaining cycle controls dynamically muxed into remaining control destinations by cycle number
- choice of doing cycle 2, 3 or 5 after decode allows saving clock cycles, may affect timing

Redrew data flow diagram:

changed sign/magnitude representation of N to 2's complement

- eliminates (8) 6LUTs used for XOR
- original purpose was to allow greater flexibility in instruction encoding

expanded block RAM address in mux to include the two 16-bit block RAM data outputs

- needed to have RTN instruction execute in three clocks
- also useful in future for load and store indirect instructions and for implementing return bit

moved CCR and IN ports of ALU mux to block RAM and made part of block RAM data out mux

- where it should be, and helps reduce data ALU complexity
- BTW, CCR not available in 16-bit width, e.g., to load or store all four CCR registers take four instructions
- additional IO via additional IO muxes

shortened data ALU delay by folding OP2/EAR mux into both data adder and into Boolean logic LUTs

- eliminates one logic delay

shortened address ALU delay by folding OP2 mux into adder along with N input and three control lines

- eliminates one logic delay

shortened CCR logic by moving overflow and all ones and all zeros logic into CCR RAM data input mux

- eliminates one logic delay
- 6 instead of 2 CCR bits to hold all zeros and all ones partial results

added fast CALL instruction (in the branch instruction subset) by restricting new PC address to PC+N or N

- saves one clock cycle
- EAR register load can be done simultaneously with new PC load

BTW: min sorted sheet does not show implementation of instruction return bit

- is shown in full sorted sheet, adds one clock cycle when invoked
- (is common in stack machine instruction sets to have a return bit in each op-code)